M1 – AAGB

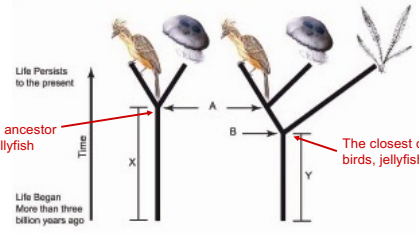
Algorithms for the reconstruction of phylogenetic trees

Alessandra Carbone
Sorbonne Université

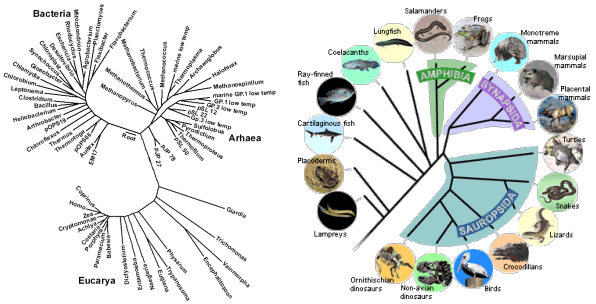
1

Reconstruction of phylogenetic trees

What are the morphological/genetical signals relating species?
Idea : compare species-specific characteristics, on the assumption that similar species are morphologically/ genetically similar.



2



Classical phylogeny: physical characteristics such as size, colour, number of legs,…. from Darwin to the early 1960s...

3

Modern phylogeny: it uses genetic information, DNA and protein sequences. Relationships between species are deduced from highly conserved blocks in the alignment of several sequences, one for each species under consideration.

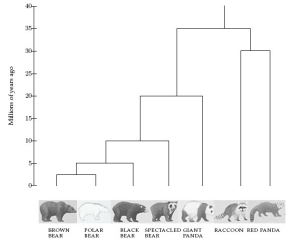
Based on sequences of 16S Ribosomal RNA for prokaryotes, 18S for eukaryotes… but also 40S, heat shock proteins… or concatenation of different proteins.

Phylogeny is not about reconstructing ancestral genomes

4

Evolution and DNA analysis: the Giant Panda riddle

- For almost 100 years scientists have been unable to understand to which family the giant panda belongs.
- Giant pandas resemble bears, but they have quite different characteristics and are closer to raccoons in their behaviour - they do not hibernate, for example.
- In 1985, Steven O'Brien et al. solved this classification problem using DNA sequences and algorithms.



- 50 years ago: Emile Zuckerkandl and Linus Pauling proposed DNA-based phylogenetic reconstruction.
- Initially, the possibility of reconstructing phylogenetic trees using DNA analysis was much debated. Today, it is the dominant approach to understanding evolution.

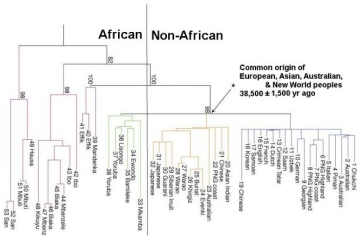
5

Out-of-Africa Hypothesis

Genetic evidence points to the African origin of all modern humans:

In the same years, when the classification of the giant panda was resolved, a DNA-based reconstruction of the human phylogenetic tree led to the Out-of-Africa Hypothesis that man's most ancient ancestor lived in Africa around 200,000 years ago.

The phylogenetic tree has separated an African group from a group containing all the 5 remaining remaining populations.

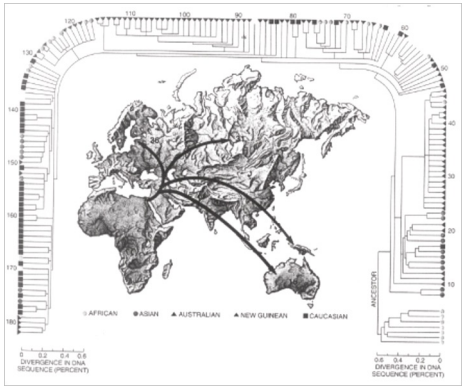


http://www.mun.ca/biology/scarr/Out_of_Africa2.htm

6

The analysis approach was based on the mitochondrial DNA of 182 individuals.

Mitochondrial DNA is particularly important because it is completely copied from mother to child, without recombination with the father's mitochondrial DNA.

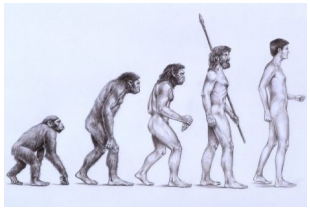


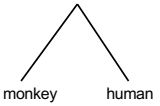
7

Phylogenetic trees

How do we build trees out of DNA sequences?

- The leaves represent existant species
- Internal nodes represent ancestors
- The root represents the oldest ancestor by the point of view of evolution.



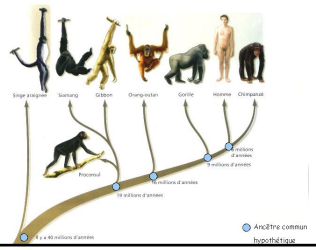


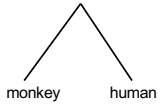
8

Phylogenetic trees

How do we build trees out of DNA sequences?

- The leaves represent existant species
- Internal nodes represent ancestors
- The root represents the oldest ancestor by the point of view of evolution.





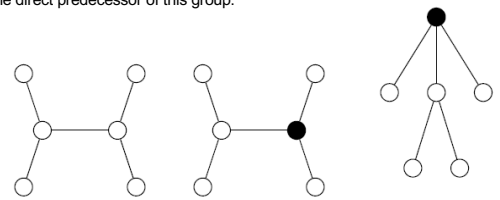
9

Rooted and unrooted trees

–A tree shows the proximity among species. It does not have a predefined root and it does not impose a hierarchy. Hence **the distance among nodes is symmetric**.

–In an unrooted tree, the position of the root ("the oldest ancestor") is not known.

–To root a tree, either we can choose a node as root or we can add a new node that will play the role of the root. Or again, we can introduce a species which is distant from all other species and the root could be the direct predecessor of this group.



(a) Unrooted tree (b) Rooted tree (c) The same rooted tree

10

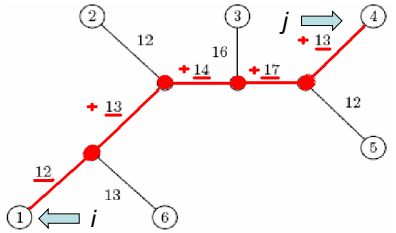
Distances in the tree

- Edges can have weights that reflect:
  - the number of mutations in the evolutionary path going from a species to the other.
  - the estimation of the evolution time going from a species to the other.
- For a tree *T*, we often compute $d_{ij}(T)$ – the *length* of the path between *i* and *j*

$d_{ij}(T)$ – ***distance*** **in the tree between *i* and *j***

11

Distance in the tree: example



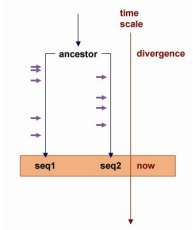
$d_{1,4} = 12 + 13 + 14 + 17 + 12 = 68$

12

To reconstruct a tree, we can make the hypothesis, that it exists a **molecular clock** governing species evolution that represents a constant rythm of the evolutionary process.

If this was the case, we could theoretically produce a phylogenetic tree preserving the distance and possibly represent the clock by an ax of time assigning to each node the moment when it appeared in the evolutionary history.

In this perfect tree, the length of each arc would be computed as the time difference between the node of the father and the node of the child.



13

There are two types of data that are used to reconstruct phylogenetic trees:

- Reconstruction based on characters : separate exam of each feature/character
- Reconstruction based on distances: the input is a distance matrix between species

14

Distance matrix

D_{ij} – ***observed distance between i and j***

Given n species, we can compute a n x n matrix of distances D_{ij}.
D_{ij} can be defined, for example, as the edit distance between a gene in the species i and the species j, where the gene belongs to the n species.

Note the difference with
$d_{ij}(T)$ – ***distance in the tree between i and j***

15

Adequate distance matrices

- Given n species, we compute the n x n **distance** matrix D_{ij}
- To visualise the evolutive relations between genes, we shall use a tree that, a priori, *we do not know,* but that we want to reconstruct.
- We need an algorithm to construct a tree that best corresponds to the distance matrix D_{ij}

16

Adequacy of the distance matrix

Ideally:

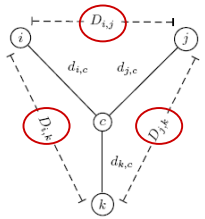
adequate means $D_{ij} = d_{ij}(T)$

$d_{ij}(T)$: length of the path in a (**unknown**) tree *T*

D_{ij}: edit distance between species (***known,*** *it is the observed distance*)

Reconstruction of a tree with 3 leaves

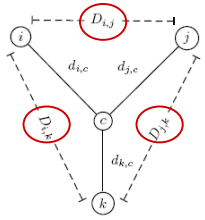
- The tree reconstruction, for all 3x3 matrices is obvious.
- We have 3 leaves *i, j, k* and a central node *c :*



Observed distances

Reconstruction of a tree with 3 leaves

- The tree reconstruction, for all 3x3 matrices is obvious.
- We have 3 leaves *i, j, k* and a central node *c :*



Observed distances

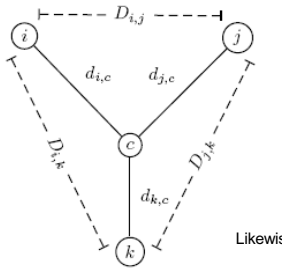
Observe: Known values

$d_{ic} + d_{jc} = D_{ij}$

$d_{ic} + d_{kc} = D_{ik}$

$d_{jc} + d_{kc} = D_{jk}$

Reconstruction of a tree with 3 leaves



$d_{ic} + d_{jc} = D_{ij}$

$+ \; d_{ic} + d_{kc} = D_{ik}$

$2d_{ic} + d_{jc} + d_{kc} = D_{ij} + D_{ik}$

$2d_{ic} + D_{jk} = D_{ij} + D_{ik}$

$d_{ic} = (D_{ij} + D_{ik} - D_{jk})/2$

Likewise,

$d_{jc} = (D_{ij} + D_{jk} - D_{ik})/2$

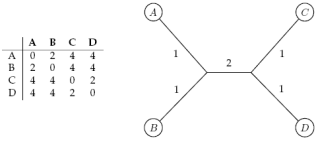
$d_{kc} = (D_{ki} + D_{kj} - D_{ij})/2$

Tree with > 3 leaves

- A tree with *n* leaves has >*2n-3* arcs
- This means that to render adequate a tree to a distance matrix *D* we shall ask for the resolution of a system of C^{n}_{2} equations with *2n-3* variables
- This is not always possible for *n* > 3

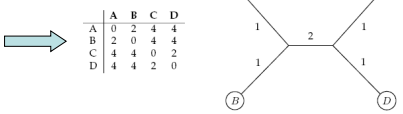
Distance matrices

The matrix *D* is ADDITIVE if it exists a tree *T* with $d_{ij}(T) = D_{ij}$

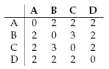


Distance matrices

The matrix *D* is ADDITIVE if it exists a tree *T* with $d_{ij}(T) = D_{ij}$



NON-ADDITIVE otherwise



?

The phylogenetic reconstruction problem based on distances

Problem: Reconstruct a phylogenetic tree starting from a distance matrix
Input: a *n* x *n* distance matrix D_{ij}
Output: a weighted tree *T* with *n* leaves representing *D*

If *D* is additive, then the problem has a solution. It exists a simple algorithm to solve it.

Use of “neighboring” leaves (leaves having a minimal number of edges connecting them) for the reconstruction of the tree

suppose that we know how to find them

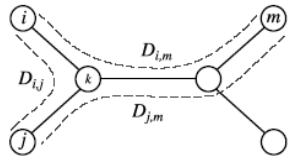
- Find the « neighboring » leaves *i* and *j* with father *k*
- Delete row and column associated to *i* and j in the matrix
- Add a new line and a new column corresponding to k, where the distance between k and all other leaves m can be computed as follows :

25

Use of “neighboring” leaves (leaves having a minimal number of edges connecting them) for the reconstruction of the tree

suppose that we know how to find them

- Find the « neighboring » leaves *i* and *j* with father *k*
- Delete row and column associated to *i* and j in the matrix
- Add a new line and a new column corresponding to k, where the distance between k and all other leaves m can be computed as follows :



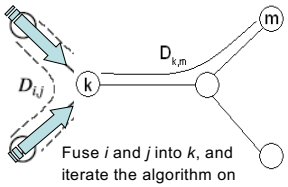
26

Use of “neighboring” leaves (leaves having a minimal number of edges connecting them) for the reconstruction of the tree

suppose that we know how to find them

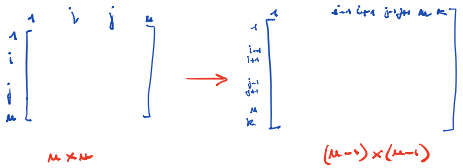
- Find the « neighboring » leaves *i* and *j* with father *k*
- Delete row and column associated to *i* and j in the matrix
- Add a new line and a new column corresponding to k, where the distance between k and all other leaves m can be computed as follows :

$$D_{km} = (D_{im} + D_{jm} - D_{ij})/2$$



Fuse *i* and *j* into *k*, and iterate the algorithm on the remaining of the tree

27



The matrix n-1 x n-1 contains the information of the matrix n x n with the exception of lines and columns I,j that have been replaced by the line/column k.

Attention : $D_{k,m} = \frac{D_{im} + D_{jm} - D_{ij}}{2}$

for all $m \neq i, j$

28

To find neighboring leaves

To find neighboring leaves we can simply select a pair of close-by leaves, that is, leaves *i* and *j* with minimal D_{ij}.

29

To find neighboring leaves

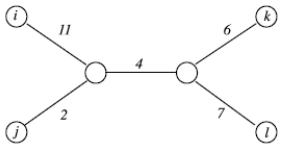
To find neighboring leaves we can simply select a pair of close-by leaves, that is, leaves *i* and *j* with minimal D_{ij}.

WRONG

30

To find neighboring leaves

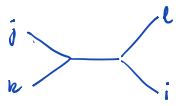
- Close-by leaves (that is, leaves *i* and *j* with minimal D_{ij}) are not necessarily neighboring leaves:
- *i* and *j* are neighboring leaves, but (d_{ij} = 13) > (d_{jk} = 12)



Based on the algorithm that we proposed, we cannot find such a tree!

31

If we choose D_{jk}=12 as neighboring leaves, we cannot construct a tree where D_{jk}= d_{jk} with weights that respect the entry matrix:

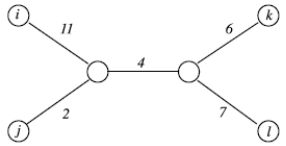


Homework:
compute the weights of the tree

32

To find neighboring leaves

- Close-by leaves (that is, leaves *i* and *j* with minimal D_{ij}) are not necessarily neighboring leaves:
- *i* and *j* are neighboring leaves, but (d_{ij} = 13) > (d_{jk} = 12)



To find pairs of neighboring leaves using only the distance matrix is not a trivial problem!

Reconstruction of trees starting from an additive matrix: a first approach

Degenerating Triplets

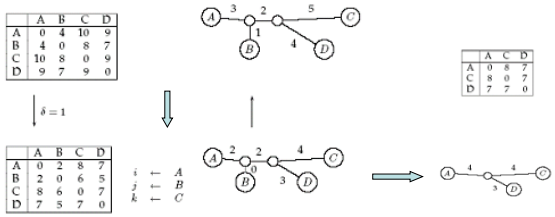
- A degenerating triplet is a set of three distinct elements $1 \le i,j,k \le n$ where $D_{ij} + D_{jk} = D_{ik}$ (usually we have $D_{ij} + D_{jk} \ge D_{ik}$)
- The element *j* in the degenerating triplet *i,j,k* is part of an evolutionary pathway from *i* to *k* (or it is attached to this path by an **edge of length 0**).

To search for degenerating triplets

- If the distance matrix *D* **has a degenerating triplet** *i,j,k* then *j* can be "eliminated" from *D* and this reduces the size of the problem.
- If the distance matrix *D* **has no degenerating triplet** *i,j,k*, then we can "create" a degenerating triplet in *D* by reducing the weight/size of all "pending edges" (edges that are connected to a leaf in the tree).

Reduction of "pending" edges to produce a degenerating triplet

Shorten all "pending" edges (that is edges that are connected to a leaf) until a degenerating triplet is found

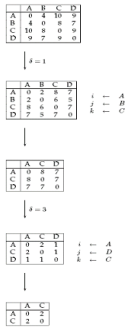


Finding degenerating triplets

- If there are no degenerating triplets, all pending edges are reduced by the same quantity δ, in such a way that all distances in the matrix are reduced by 2δ.
- At the end, this procedure collapses one of the leaves (when δ = length of the shortest « pending » edge), it forms a degenerating triplet *i,j,k* and it reduces the size of the distance matrix *D*.
- The attachement point for *j* can be recovered in the reversed transformations by saving D_{ij} at each collapsed leaf.

Reconstruction of trees for matrices with additive distance

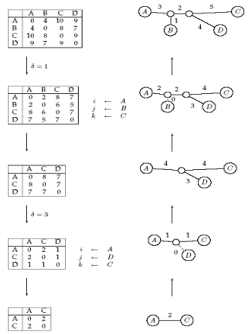
$D_{ij} + D_{jk} = D_{ik}$



37

Reconstruction of trees for matrices with additive distance

$D_{ij} + D_{jk} = D_{ik}$



38

AdditivePhylogeny Algorithm

1. **AdditivePhylogeny**(D)
2. **if** D is a 2×2 matrix
3. T = tree made of a single edge of length $D_{1,2}$
4. **return** T
5. **if** D is non-degenerated
6. δ = reduction parameter of the matrix D
7. **for all** $1 \le i \ne j \le n$
8. $D_{ij} = D_{ij} - 2\delta$
9. **else**
10. $\delta = 0$

39

AdditivePhylogeny

1. Find a triplet i, j, k in D such that $D_{ij} + D_{jk} = D_{ik}$
2. $x = D_{ij}$
3. Delete row j^{th} and column j^{th} of D
4. T = AdditivePhylogeny(D)
5. Add a new node v to T at distance x from i to k
6. Add j to T by creating an edge (v,j) of length 0
7. **for each leaf** l in T
8. **if the distance from** l to v in the tree $\ne D_{l,j}$
9. output "the matrix is not additive"
10. **return**
11. Extend all "pending" edges of a length δ
12. **return** T

40

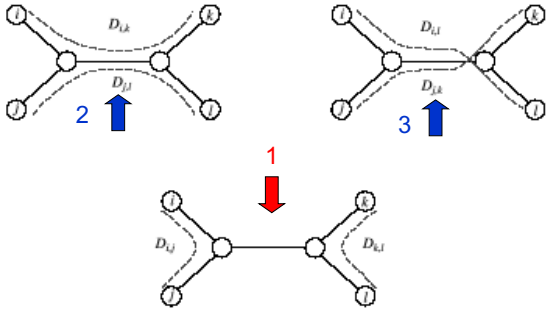
The four points condition

- AdditivePhylogeny provides a way to test if the distance matrix *D* is additive
- **A better additivity test is based on the "4 points condition"**
- Let $1 \le i,j,k,l \le n$ be four distinct leaves in the tree.

41

The four points condition

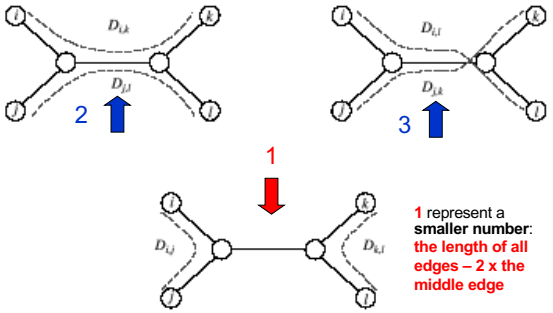
Compute: 1. $D_{ij} + D_{kl}$, 2. $D_{ik} + D_{jl}$, 3. $D_{il} + D_{jk}$



42

The four points condition

Compute: 1. $D_{ij} + D_{kl}$, 2. $D_{ik} + D_{jl}$, 3. $D_{il} + D_{jk}$



2 and **3** represent the **same number**: the length of all edges + the middle edge (it is counted twice)

1 represent a **smaller number**: **the length of all edges – 2 x the middle edge**

43

The four points condition : theorem

- The 4 points condition for the quartet *i,j,k,l* holds if two of the sums are the same, and the third sum is smaller than the first two.
- **Theorem** : A *n* x *n* matrix *D* is additive iff
  the 4 points condition is true for **all** quartets *i,j,k,l*, where $1 \le i,j,k,l \le n$.

44

Least Squares Distance Phylogeny Problem

- Si la matrice de distance *D* n'est pas additive, alors on cherche un arbre *T* que approxime *D* pour le mieux:

  Squared Error : $\sum_{i,j} (d_{ij}(T) - D_{ij})^2$

- Squared Error est une mesure de la qualité de l'adéquation entre matrice de distance et arbre: on veut la minimiser.
- **Least Squares Distance Phylogeny Problem**: trouver le meilleur arbre approximé *T* pour une matrice non-additive *D* (NP-hard).

On présentera dans la suite deux algorithmes euristiques polynomiaux, **UPGMA** et **Neighbor-Joining**.

Least Squares Distance Phylogeny Problem

- If the distance matrix *D* is not additive, then we search for a tree *T* that approximates *D* for the better:

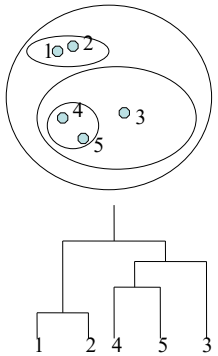
  Squared Error : $\sum_{i,j} (d_{ij}(T) - D_{ij})^2$

- Squared Error is a measure of the quality of the adequation between the distance matrix and a tree: we want to minimize it.
- **Least Squares Distance Phylogeny Problem**: find the best approximating tree *T* for the non-additive matrix *D* (NP-hard).

We shall present two polynomial algorithms based on heuristics, UPGMA and Neighbor-Joining.

UPGMA: Unweighted Pair Group Method with Arithmetic Mean

UPGMA is a clustering algorithm:
- It computes the distance between clusters by using the average distance among pairs
- It assigns a *height* to each node in the tree, under the assumption of an existing molecular clock.



Clusterisation by UPGMA

Given two disjoint clusters C_i, C_j of species,

$$d_{ij} = \frac{1}{|C_i| \times |C_j|} \sum_{\{p \in Ci,\ q \in Cj\}} d_{pq}$$

Average distance of Ci, Cj computed on all pairs

Note that if $C_k = C_i \cup C_j$, then the distance between C_k and another cluster C_l is:

$$d_{kl} = \frac{d_{il}\,|C_i| + d_{jl}\,|C_j|}{|C_i| + |C_j|}$$

Weighted average of distances between clusters

UPGMA algorithm

Initialisation:

Assign each x_i to its proper cluster C_i

Define a leaf per species, each one at height 0.

Iteration:

Find two clusters C_i and C_j such that d_{ij} is minimal

Let $C_k = C_i \cup C_j$

Add a node connecting C_i, C_j and place it at a height $d_{ij}/2$

Suppress C_i and C_j

Compute the distance between the new cluster C_k and all other clusters as a weighted average of the distances between the components.

Termination:

When we obtain only one cluster.

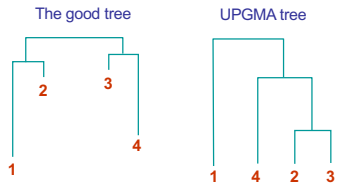
The complexity of UPGMA is $O(n^2)$: there are n-1 iterations, with O(n) operations for each of them.

Weakenesses of UPGMA

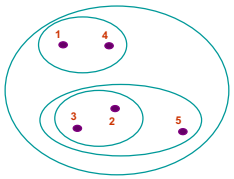
The algorithm produces an **ultrametric** tree : the distance from the root to the leaves is the same. This means that there exists a **molecular clock** having a constant rythm: each species represented by some leaf in the tree are thought as having accumulated mutations (and therefore as having evolved) with the same mutation rate.

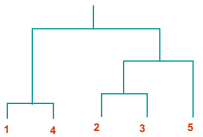
This is the major problem of UPGMA.

For instance:



It works under the assumption that clusters once fused are close, one to the other, without asking them to be far from the rest.





Neighbor Joining Algorithm

- In 1987, Naruya Saitou and Masatoshi Nei developed an algorithm of « neighbor joining » for the reconstruction of phylogenetic trees.
- **Find a pair of leaves that are close to each other but far from the remaining leaves:** implicitly we should find a pair of leaves in a « neighborhood ». To each iteration, the algorithm tries to find the direct ancestor of two species in the tree. For each node i, its distance u_i from the rest of the tree is estimated by using the formula

$$u_i = \sum_{k \neq i} \frac{D_{i,k}}{n-2}$$

To minimise the sum of all branching lengths, the nodes i and j that are clustered are those with

$D_{i,j} - u_i - u_j$ minimised

- The lengths $d_{k(ij)}$ of the new branches are computed by solving the system of linear equations that gives all distances between pairs (see step 6 in the algorithm).

Avantages: it works well on additive matrices and on certain non-additive matrices; it does not consider the molecular clock hypothesis.

NJ Algorithm

Initialisation:

1. Initialize n clusters with the different species, a species per cluster
2. Fix the size of the clusters to 1, for each cluster
3. on the resulting tree T, associate a leaf to each species, with a height 0

Iteration:

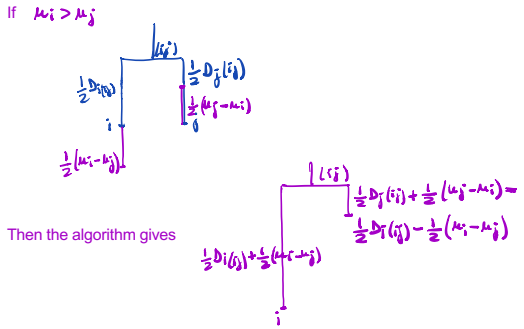
4. to each species, compute $u_i = \sum_{k \neq i} \frac{D_{i,k}}{n-2}$
5. choose i and j such that $D_{i,j} - u_i - u_j$ is minimal
6. join the clusters i et j in a new cluster (ij), with a corresponding node in T. Compute the length of the branches from i and j to the new nodes as follows:

$$d_{i(ij)} = \tfrac{1}{2} D_{ij} + \tfrac{1}{2} (u_i - u_j) \qquad d_{j(ij)} = \tfrac{1}{2} D_{ij} + \tfrac{1}{2} (u_j - u_i)$$

7. Compute the distances between the new cluster and all other clusters

$$D_{(ij)k} = (D_{ik} + D_{jk} - D_{ij}) / 2$$

8. Suppress clusters i and j from the tables, and replace them with (ij)
9. If more than 2 nodes (clusters) remain, go back to 4. Otherwise, connect the two remaining nodes with an edge of length D_{ij}.



The complexity of the algorithm NJ is $O(n^3)$
(more performing versions have been proposed)

Reconstruction of the tree based on characters

Problem (optimal phylogenetic tree)

Input:
- A set of n species
- A set of m characters concerning all species
- For each species, the value of each character
- Optimal parameters that are dependent on the problem

Output: a phylogenetic tree which is completely labeled that explains the data, that is, that maximises a given target function.

Note : the characters can be nucleotides, where A, G, C, T are the states of the character. Other characters can be the # of eyes or the # of legs or forms of ...

Assumptions:

1 Characters are mutually independent, that is a changement in a character has no effect on the distribution of another character.

2 When two species have diverged in the tree, they continue to evolve independently.

Probably these assumptions are not correct, but they simplify the analysis.

A **trivial solution** to the phylogeny problem is to simply enumerate all possibles trees and compute the value of a « target function » for each tree.

Under the assumption that phylogenetic trees are binary trees, the number of trees that are non-isomorphic, labelled, binary, rooted and containing n leaves is :

$$1 * 3 * 5 * \ldots * (2n-3) = \prod_{i=2}^{n} (2i-3) = (2n-3)!! = \Omega((2n/3)^{n-1})$$

that is super-exponential. For n=20, we have around 10^{21} such trees.

The same is true for the number of unrooted trees

$$= (2n-5)!! = \Omega((2n/3)^{n-2})$$

(Cavalli-Sforza et Edwards, 1967)

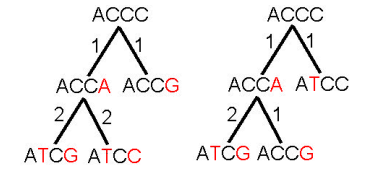
Parsimony approach to the reconstruction of a phylogenetic tree

- Apply the principle of Occam razor – that is identify the simpler explanation for the data.
- Make the hypothesis that the differences between observed characters are the result of a minimal number of mutations.
- Look for the tree that corresponds to the smaller **parsimony score** – sum of the costs of all mutations found in the tree.

Parsimony and tree reconstruction

Fixing the length of an edge in the tree by using the Hamming distance, we can define the parsimony score of a tree of sequences as the sum of the lengths (weights) of the egdes.

Characters are positions in the sequence.



Moins parcimonieux
Score: 6

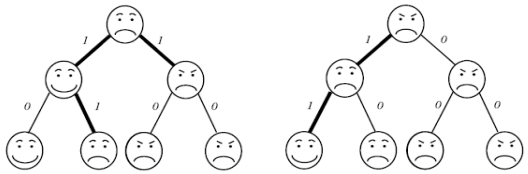
Plus parcimonieux
Score: 5

The approach minimising scores is called parsimony

61

A second example of tree reconstruction : two characters

(eyebrows mouth) : we look for words of two letters



(a) *Parsimony Score=3* (b) *Parsimony Score=2*

Figure 10.16 If we label a tree's leaves with characters (in this case, eyebrows and mouth, each with two states), and choose labels for each internal vertex, we implicitly create a *parsimony* score for the tree. By changing the labels in (a) we are able to create a tree with a better parsimony score in (b).

62

Problem 1 : the «small parsimony problem»

Input : a tree *T* with all its leaves labeled by a word of m characters

Output : a labeling of the internal nodes of the tree *T* through a minimisation of the parsimony score

We can make the hypothesis that each leaf is labeled by exactly one character, because characters in words are independent.

63

Problem 2 : problem of weighted parsimony

It is a more general version of the «small parsimony problem».

- The input considers a *k* * *k* score matrix that describes the transformation cost of each one of the *k* states into another.
- Recall that for the small parsimony problem, the score matrix is defined by the Hamming distance

  $d_H(v, w) = 0$ si $v=w$

  $d_H(v, w) = 1$ sinon

  character values

64

Score matrices

Small Parsimony Problem

	A	T	G	C
A	0	1	1	1
T	1	0	1	1
G	1	1	0	1
C	1	1	1	0

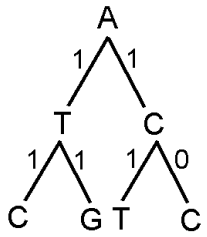
Weighted Parsimony Problem

	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0

Weighted parsimony matrix

65

Non-weighted vs. weighted



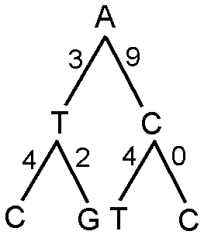
Score matrix of small parsimony:

	A	T	G	C
A	0	1	1	1
T	1	0	1	1
G	1	1	0	1
C	1	1	1	0

Score of small parsimony : 5

66

Non-weighted vs. weighted



Score matrix of weighted parsimony:

	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0

Weighted parsimony score : 22

67

The weighted parsimony problem

Input: a tree *T* (defined by its topology and its root) with all leaves labelled by the elements of an alphabet of k letters and a *k* x *k* score matrix δ_{ij}

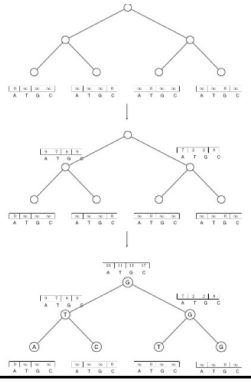
Output: a labeling of internal nodes of a tree T minimising the weighted parsimony score.

68

Sankoff algorithm (1)

- Test all children of each node v and determine the minimal cost of the subtree with root v
- An example:

δ	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0



69

Sankoff algorithm: dynamic programming

Etape 1:

- For each node in the tree, compute and keep the score associated to each possible label
  - $s_t(v)$ = score of minimal parsimony of the subtree rooted at node v, if v is a character with value t
- The score to each node is based on the score of its children :
  - s_t(***père***) = $\min_i \{s_i($ ***fils gauche*** $) + \delta_{i,t}\}$ + $\min_j \{s_j($ ***fils droit*** $) + \delta_{j,t}\}$

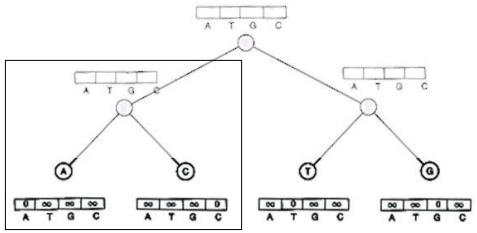
  where t,i,j are indices on the different values of a character and
  $\delta_{i,t}$ represents the cost of the transformation of a value i associated to a child into a value t associated to the father

70

Sankoff algorithm (2)

We start by considering the leaves:

- if a leaf has a character then the score is 0
- otherwise the score is ∞



71

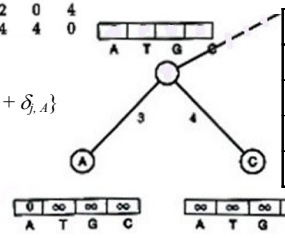
Sankoff algorithm (3)

Cost of the transformation

$$s_t(v) = \min_i \{s_i(u) + \delta_{i,t}\} + \min_j \{s_j(w) + \delta_{j,t}\}$$

δ	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0

$$s_A(v) = 0 + \min_j \{s_j(w) + \delta_{j,A}\}$$



	$s(u)$	$\delta_{i,A}$	sum
A	0	0	**0**
T	∞	3	∞
G	∞	4	∞
C	∞	9	∞

72

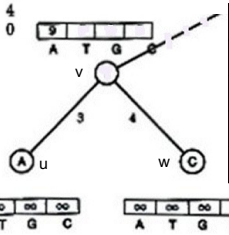
Sankoff algorithm (4)

Cost of the transformation

$s_t(v) = \min_i \{s_i(u) + \delta_{i,t}\} + \min_j \{s_j(w) + \delta_{j,t}\}$

δ	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0

$s_A(v) = 0 + 9 = \mathbf{9}$



	$s_i(u)$	$\delta_{i,A}$	sum
A	∞	0	∞
T	∞	3	∞
G	∞	4	∞
C	0	9	**9**

73

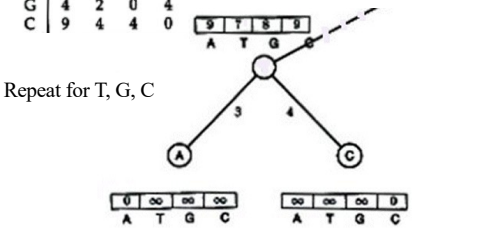
Sankoff algorithm (5)

Cost of the transformation

$s_t(v) = \min_i \{s_i(u) + \delta_{i,t}\} + \min_j \{s_j(w) + \delta_{j,t}\}$

δ	A	T	G	C
A	0	3	4	9
T	3	0	2	4
G	4	2	0	4
C	9	4	4	0

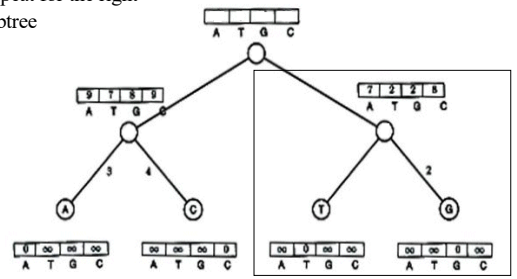
Repeat for T, G, C



74

Sankoff algorithm (6)

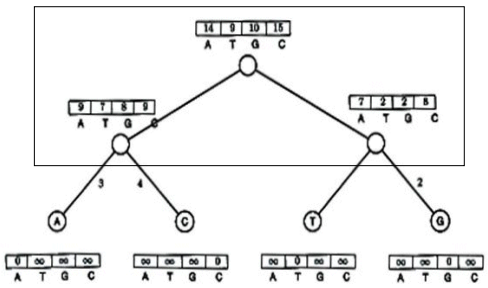
Repeat for the right subtree



75

Sankoff algorithm (7)

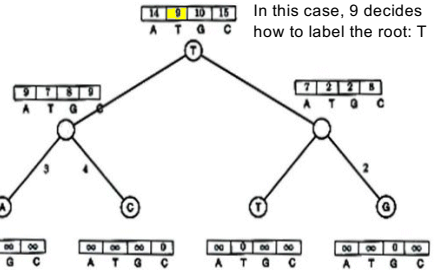
Repeat for the root



76

Sankoff algorithm (8)

The smallest score on the root is the minimal weighted parsimony score



77

Sankoff algorithm: going from the root to the leaves

Step 2:

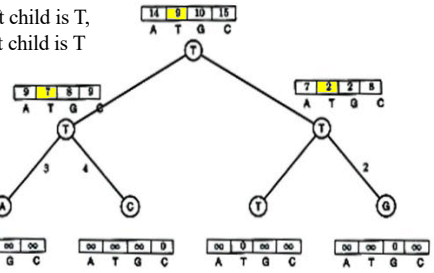
- Scores at the root have been computed going up the tree from the leaves.
- When the score at the root is computed, the algorithm backtracks and assignes an optimal character to each node it encounters.

78

Sankoff algorithm (9)

9 is derived from 7 + 2

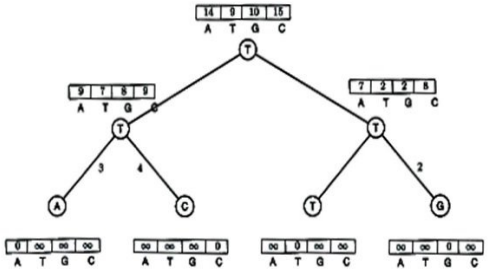
Then the left child is T, and the right child is T



79

Sankoff algorithm (10)

And the tree is labelled…



80

Fitch algorithm

It is a dynamic programming algorithm that solves the small parsimony problem :

Step 1:

Assign a set of letters to each node of the tree:

- If the two sets of characters associated to the children overlap, then consider their intersection.
- Otherwise compute the union of the sets. (For instance, if the node that we examine has the left child labelled {A, C} and the right child labelled {T}, then the node will be labelled by {A, C, T})

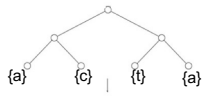
Fitch algorithm

Step 2:

Assign labels to each node by traversing the tree from the root to the leaves

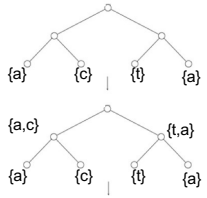
- Assign the root in an arbitrary manner by choosing within the set of letters determined at step 1.
- For all other node, if the label of the father is within the set of letters associated tot he node, assign to it the label of the father.
- Otherwise, choose arbitrarily a letter within the set of possible labels.

Fitch algorithm (1)



An example:

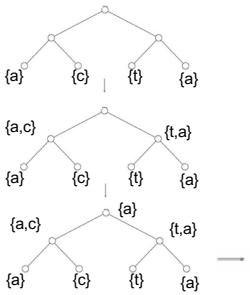
Fitch algorithm (1)



An example:

Fitch algorithm (1)

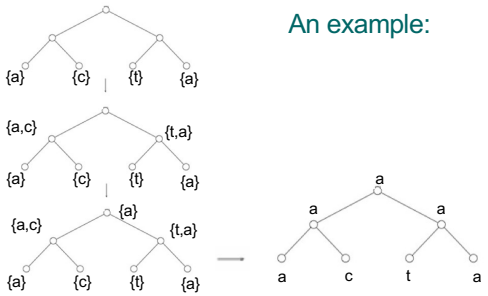
An example:



85

Fitch algorithm (1)

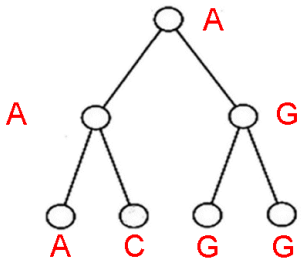
An example:



86

Fitch algorithm (2)

Another example :



87

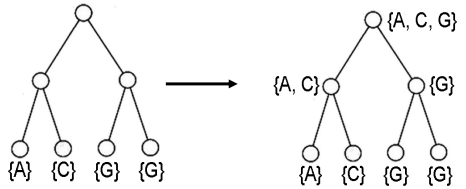
Fitch vs. Sankoff

- The two algorithms have time O(*nk*) (where n = species and k = letters))
- Are they really different ?
- We compare them …

88

Fitch

As seen before:



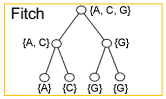
with the score matrix (for Fitch algorithm) that is simply:

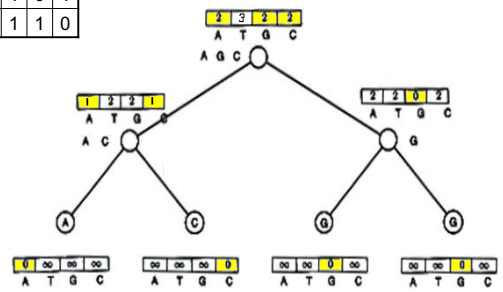
	A	T	G	C
A	0	1	1	1
T	1	0	1	1
G	1	1	0	1
C	1	1	1	0

We shall solve the same problem with the Sankoff algorithm and the same score matrix:

Sankoff

	A	T	G	C
A	0	1	1	1
T	1	0	1	1
G	1	1	0	1
C	1	1	1	0





Sankoff vs. Fitch

- Sankoff algorithm given the **same** set of **optimal** labels than the Fitch algorithm.
- For Sankoff algorithm, a character *t* is *optimal* for a node *v* if $s_t(v) = \min_{1 \le i \le k} s_i(v)$
  - Denote $S(v)$ the set of optimal letters at node *v:*
    - if $S(left\ child)$ and $S(right\ child)$ overlap, $S(father)$ is the intersection
    - otherwise, make the union of $S(left\ child)$ et $S(right\ child)$

    But this is also Fitch recurrence
- The two algorithms are **identical** (on score matrices 0/1)